

Onde se concentram as vulnerabilidades que já estão sendo exploradas

Data lake na AWS provisionado 100% em Terraform sobre o catálogo público CISA KEV, com custo de consulta medido.

Grupo g07 · CESAR School · 2026.2

Integrantes: Bruno Ribeiro · Emanuel Eduardo · Igor Alves

Leonardo Amaral · Victor Guilherme · Vinícius Petribu

Cenário e pergunta de negócio

- A **CISA** publica o catálogo **KEV**: vulnerabilidades com exploração **confirmada em ataques reais** — não teóricas.
- Toda equipe de segurança enfrenta a mesma escassez: **mais CVEs do que capacidade de corrigir**.
- A pergunta que decide a fila de correção: **quais fabricantes concentram as vulnerabilidades exploradas nos últimos 12 meses?**
- E a pergunta que decide a arquitetura: **quanto custa responder isso no Athena?**

1.713

CVEs no catálogo
(catalogVersion 2026.09.16)

299

entraram nos últimos 12 meses

118

fabricantes distintos na janela

Arquitetura provisionada

CISA KEV — feed público

JSON, sem autenticação, 1.727.984 bytes

▼ ingest.sh — 5 barreiras de qualidade

S3 · eda262-g07-lake-raw

documento original, byte a byte — prova de origem

▼ limpeza e tipagem → NDJSON

S3 · eda262-g07-lake-trusted

1.713 linhas · 1.502.572 bytes

▼ schema declarado em IaC

Glue Data Catalog · eda262_g07_kev

kev_vulnerabilities — 15 colunas, sem Crawler



Athena WorkGroup · eda262-g07-wg

limite de 100 MB por consulta, imposto no WorkGroup

- **100% Terraform**, empacotado como módulo.
- Backend remoto **S3 + DynamoDB**, com workspace.
- Tags obrigatórias via **default_tags** do provider — recurso novo não consegue esquecê-las.
- **Zero Glue Crawler**: o schema é código revisável.

Granularidade: uma linha por CVE

- Grão declarado **no próprio catálogo**, como parâmetro da tabela:

```
grain = "one row per cve_id"
```

- **Provado, não afirmado:** uma named query roda **count(*)** – **count(DISTINCT cve_id)** e precisa retornar 0.
- Alternativa avaliada e **descartada com número**: um grão por (CVE, produto).
- **214 de 1.713** valores de **product** (12,5%) trazem separador ambíguo.

```
"NetScaler ADC and NetScaler Gateway" → 2 produtos  
"Community Edition and Enterprise Edition" → 1 produto
```

Nenhuma regra determinística separa os dois casos. Dividir pelo separador inflaria a contagem de produtos por artefato de parsing — no indicador que a pergunta mede.

1.713

linhas na tabela trusted

1.713

cve_id distintos

0

duplicatas — o grão se sustenta

Da camada raw para a trusted

PROBLEMA MEDIDO NA ORIGEM	ANTES	DEPOIS
Espaço espúrio em vendor / product	18	0
"Known" / "Unknown" como texto	360 / 1.353	boolean
"Yes" / "No" como texto	55 / 1.658	boolean
cwes sem contagem pré-calculada	array	int
1 objeto JSON com array aninhado	1 linha	1.713 linhas

A ingestão só publica se as **5 barreiras** passarem. A barreira de grão bloqueia o upload — dado que viola a modelagem não chega à camada trusted.

POR QUE ISSO NÃO É COSMÉTICO

- O valor **"SimpleHelp "** (com espaço) aparece em **4** registros.
- Sem **TRIM**, uma ocorrência futura sem espaço criaria **dois grupos** no GROUP BY.
- Seria a diferença entre um ranking correto e um **silenciosamente errado**.

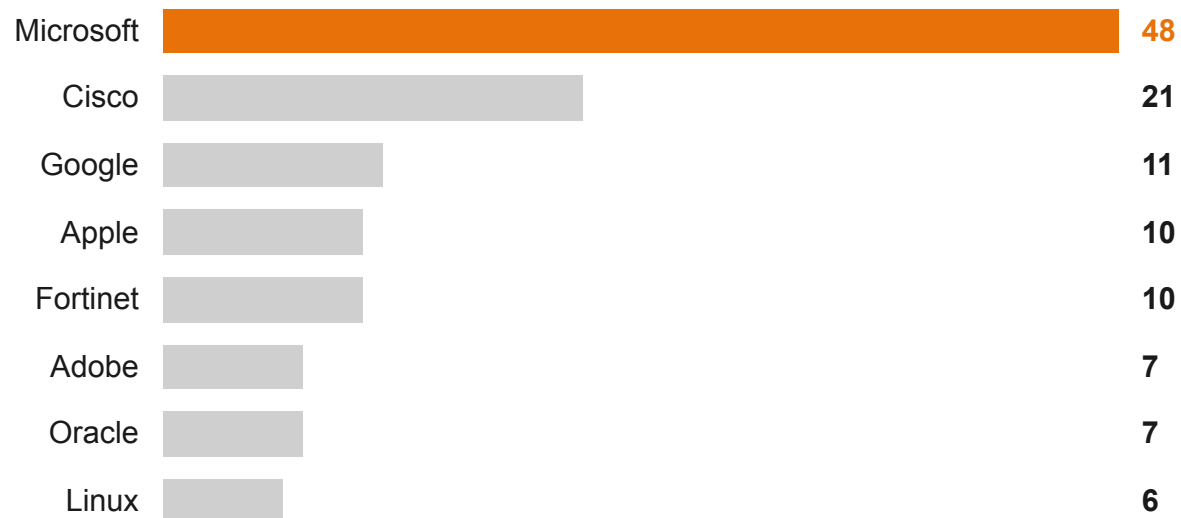
Schema declarado em código, sem Crawler

- As **15 colunas** são declaradas em Terraform e renderizadas a partir de **uma única fonte de verdade**.
- Um Crawler custa **USD 0,073** por execução (USD 0,44/DPU-hora, mínimo de 10 min — preço de tabela).
- Sobre um schema estável, ele pagaria **para redescobrir o que já sabemos** — e poderia variar o tipo entre execuções.
- Schema como código custa **USD 0,00**, entra em **code review** e falha no **plan**, não em produção.

```
columns = [  
  { cve_id           string }  
  { vendor_project   string }  
  { product          string }  
  { date_added       string }  
  { is_known_ransomware_use boolean }  
  { requires_forensic_triage boolean }  
  { cwes             array<string> }  
  { cwe_count        int }  
  { catalog_version  string }  
  { ingested_at      string }  
  ...  
]
```

Datas ficam **string** em ISO-8601: em NDJSON todo valor é texto, e forçar **date** transformaria falha de parsing em **NULL silencioso** — justamente no filtro de 12 meses.

A resposta: concentração nos últimos 12 meses



CVEs adicionados ao KEV nos últimos 12 meses, por fabricante. Base: 299 CVEs. Top 1 = 16% · top 5 = 33% · top 10 = 44%. Fonte: [docs/evidence/business-question-result.csv](#).

O ACHADO QUE MUDA A FILA

10% vs 43%

proporção ligada a **ransomware**:
Microsoft (5 de 48) · Oracle (3 de 7)

Volume **não** é risco. Quem corrige só pelo topo do ranking trata a Oracle como sétima prioridade — sendo que quase metade das KEV dela já foi usada em ransomware.

Custo medido da consulta

MEDIÇÃO	VALOR
Tamanho da tabela trusted	1.502.572 B
Volume varrido (medido)	3.005.144 B
Mínimo cobrado pelo Athena	10.485.760 B
Volume efetivamente cobrado	10.485.760 B
Preço (us-east-1)	USD 5,00 / TB
Consultas por USD 1,00	≈ 20.973

USD 0,00004768

custo por execução da consulta de negócio

POR QUE NÃO USAMOS PARQUET

- A consulta varre **3,0 MB** e cabe **3,5×** dentro do mínimo cobrável de 10 MB.
- A varredura é **2,00× a tabela**: a subconsulta escalar faz o Athena ler os dados duas vezes — só descoberto medindo.
- Parquet e particionamento reduziriam os **bytes lidos** — e **nenhum centavo** da fatura, que já está no piso.
- Economia medida da otimização: **USD 0,00**.
- Entra na Parte 2, quando o volume passar do piso — decisão por número, não por calendário.

Disciplina de infraestrutura

- **Módulo + backend remoto + workspace.** O workspace **av1** gera os nomes exigidos; outros recebem sufixo, porque nome de bucket S3 é **global**.
- **O workspace default é bloqueado** por precondition: falha no plan, antes de criar qualquer recurso.
- **O SQL mora no Terraform** como named query, e o script a executa **por ID** — o que roda não pode divergir do que foi revisado.
- **Destroy é verificado, não presumido:** o Terraform retorna sucesso mesmo deixando órfão. Conferimos os **5** recursos na AWS depois.

GUARDRAIL DE CUSTO

100 MB

limite por consulta, imposto no WorkGroup — $\approx 70\times$ o dataset atual

25

critérios em verifica.sh, com PASSA / FALHA por item

0

recursos órfãos tolerados após o destroy

Cada decisão, um número

DECISÃO	NÚMERO QUE A SUSTENTA
Grão: uma linha por CVE	1.713 / 1.713 · 0 duplicatas
Grão por produto descartado	214 ambíguos (12,5%)
Limpeza na camada trusted	18 → 0
Schema em IaC, sem Crawler	15 colunas · USD 0,073 evitados
Custo por consulta	USD 0,00004768
Parquet adiado para a Parte 2	economia de USD 0,00
Destroy sem órfãos	5 recursos conferidos

PRÓXIMA FASE

- Volume que **ultrapasse o piso** de cobrança — só então Parquet e particionamento têm economia mensurável.
- Ingestão **idempotente** com backfill de partição.
- Camada refined e permissões por camada via **Lake Formation**.

Repositório: eda262-projeto-g07

Evidências: docs/evidence/

Decisões: DECISOES.md